备，或者 register_netdev() 注册网卡。

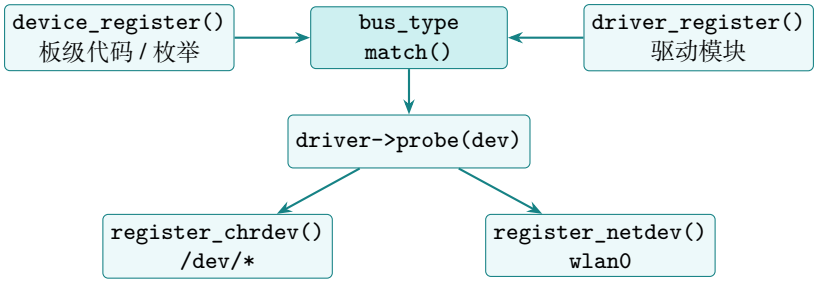



图 7.1: 设备与驱动的绑定

项目中有三条总线：

- **platform**：SoC 内固定的设备（Mini UART、SDHOST、Arasan、DWC2），无法枚举，由板级代码静态描述，按名字匹配；
- **usb**：DWC2 枚举出的 USB 设备，按 VID/PID 或接口类别匹配；
- **sdio**：Arasan 枚举出的 SDIO function，按 vendor/device/function 匹配。

字符设备的主设备号见表 7.1。设备节点由内核的 devtmpfs 按已注册的驱动自动创建（do_mounts.c），init 只在 devtmpfs 不可用时才用 mknod 补建；cat /proc/devices 可以看到全部总线、设备、资源和已绑定的驱动。

表 7.1: 字符设备主设备号

| major | 名称 | 节点 |
|---|---|---|
| 1 | console | /dev/console |
| 2 | tty（当前进程的控制终端） | /dev/tty |
| 3 | ttyS0（Mini UART） | /dev/ttyS0 |
| 4 | input（evdev） | /dev/input/event0 |
| 5 | camera | /dev/video0 |

### 7.1.2 kobject：一切皆对象

上面对象只是“数据”，要在运行时看见并操作它们，就得把它们挂上一棵统一对象树，再由一棵伪文件系统把树渲染成目录与文件。Linux 用 kobject（lib/kobject.c）与它的渲染器 sysfs（fs/sysfs）完成此事，本工程照搬同一分层（kernel/kobject.c、kernel/vfs.c）。

/sys 里没有“设备目录”“驱动文件”的特判：唯一的节点类型就是 struct kobject——一个带名字、父指针、引用计数与 ktype 的对象。bus_type、device、device_driver 各自**内嵌**一个 kobject 并给出一个 kobj_type，于是 /sys/bus、/sys/devices、驱动目录全都出自同一个通用遍历器：

```
struct attribute { char *name; int mode; };      // 一个 sysfs 文件
struct sysfs_ops {
  int (*show)(struct kobject*, struct attribute*, char *buf, int size);
  int (*store)(struct kobject*, struct attribute*, char *buf, int size);
};
struct kobj_type {
  void (*release)(struct kobject*);               // refcount 归零时回收
  struct sysfs_ops *sysfs_ops;                    // show/store 分派
  struct attribute **default_attrs;               // 该类对象的默认属性
};
struct kobject {
  char name[KOBJ_NAME_MAX];  struct kobject *parent;
  struct kobj_type *ktype;   struct attribute **attrs; // 实例属性叠加在默认之上
```